Understanding Syntax: Python and R Compared

A Comparative Introduction for Data Analysis

Data Analysis for Decision-Making

2026-04-28

Why Compare Python and R?

> *"The goal is not to choose a language — it is to understand data. (Manus Ai)"*

- Both **R** and **Python** are industry-standard tools for data analysis and statistical computing
- **R** was designed specifically for statistical analysis and data visualisation
- **Python** is a general-purpose language widely adopted in data science, machine learning, and automation
- This presentation focuses on **syntax** — the grammar of each language — to help you read, write, and translate code confidently

Python and R at a Glance

Feature	R	Python
Primary purpose	Statistical computing & graphics	General-purpose programming
Key strength	Statistics, visualisation (ggplot2)	ML, automation, versatility
Main data tool	data.frame / tibble	pandas DataFrame
Package ecosystem	tidyverse, ggplot2	pandas, scikit-learn
Index start	1 (one-based)	0 (zero-based)

Shared Foundations: What Python and R Have in Common

- Both are **interpreted** languages
- Both support **vectorised operations**
- Both integrate with **Quarto / R Markdown / Jupyter** for reproducible reporting
- Both support **functional programming**
- Both use the **data frame** as the primary tabular data structure

R — Vectorised Operation

```r
scores <- c(85, 90, 78, 92)
scaled <- scores / 100
print(scaled)
# [1] 0.85 0.90 0.78 0.92
```

Python — Vectorised Operation

```python
import numpy as np
scores = np.array([85, 90, 78, 92])
scaled = scores / 100
print(scaled)
# [0.85 0.90 0.78 0.92]
```

Assignment and Variables: Different Conventions, Same Logic

R — Assignment

```r
# Primary assignment operator: <-
student_name <- "Srikumar"
score        <- 92.5
is_passed    <- TRUE

# Print by typing the name
student_name
# [1] "Srikumar"
```

- Uses `<-` (preferred) or `=`
- Prints by simply typing the variable name
- Dots allowed in names: `my.var`

Python — Assignment

```python
# Only assignment operator: =
student_name = "Srikumar"
score        = 92.5
is_passed    = True

# Explicit print required in scripts
print(student_name)
# Srikumar
```

- Uses `=` exclusively
- Requires `print()` in scripts
- Underscores standard: `my_var`

Data Structures: Vectors, Lists, and Data Frames

R — Data Structures

```r
# Vector: same-type elements
grades <- c(85, 90, 78)

# Named list: mixed types
student <- list(
  name  = "Srikumar",
  grade = 90
)

# Data frame: tabular data
df <- data.frame(
  name  = c("Alice", "Bob"),
  score = c(88, 76)
)
```

Python — Data Structures

```python
import pandas as pd

# List: ordered, mixed types
grades = [85, 90, 78]

# Dictionary: key-value pairs
student = {
  "name":  "Srikumar",
  "grade": 90
}

# DataFrame: tabular data
df = pd.DataFrame({
  "name":  ["Alice", "Bob"],
  "score": [88, 76]
})
```

> **Note:** In R, data frames are built on vectors, while in Python, DataFrames are provided through the pandas library.

Indexing and Filtering: 1-Based vs. 0-Based

R — Indexing & Filtering

```r
grades <- c(85, 90, 78, 92)

# 1-based: first element is [1]
grades[1]       # 85
grades[2:3]     # 90 78

df <- data.frame(
  name  = c("Alice","Bob","Carol"),
  score = c(88, 76, 95)
)

# Filter rows where score > 80
df[df$score > 80, ]
```

Python — Indexing & Filtering

```python
import pandas as pd

grades = [85, 90, 78, 92]

# 0-based: first element is [0]
grades[0]       # 85
grades[1:3]     # [90, 78]

df = pd.DataFrame({
  "name":  ["Alice","Bob","Carol"],
  "score": [88, 76, 95]
})

# Filter rows where score > 80
df[df["score"] > 80]
```

> **Key rule:** R starts counting at **1**; Python starts at **0**. Slicing in Python excludes the end index.

Translating R into Python: A Complete Workflow

R — Student Score Analysis

```r
scores <- c(72, 85, 90, 88,
            76, 95, 60, 83)

mean_score <- mean(scores)
sd_score   <- sd(scores)

cat("Mean:", mean_score,
    "\nSD:", sd_score, "\n")

# Filter: above average
above_avg <- scores[scores > mean_scor
print(above_avg)
```

Python — Equivalent Code

```python
import numpy as np

scores = np.array([72, 85, 90, 88,
                   76, 95, 60, 83])

mean_score = np.mean(scores)
sd_score   = np.std(scores, ddof=1)

print(f"Mean: {mean_score:.2f}"
      f"\nSD: {sd_score:.2f}")

# Filter: above average
above_avg = scores[scores > mean_score
print(above_avg)
```

Translation map:

R	Python
`mean()`	`np.mean()`
`sd()`	`np.std(ddof=1)`
`cat()`	`print(f"...")`
`scores[condition]`	`scores[condition]`

R in Action: Exploratory Data Analysis with dplyr

Scenario: Analyse MBA student performance by programme using R's `dplyr` pipeline.

```r
library(dplyr)

# Sample dataset
students <- data.frame(
  name    = c("Alice", "Bob", "Carol", "David", "Eva"),
  score   = c(88, 72, 95, 65, 83),
  program = c("Finance", "Marketing", "Finance", "HR", "Marketing")
)

# Analysis pipeline using the pipe operator |>
summary_stats <- students |>
  filter(score >= 70) |>                    # keep students who passed
  group_by(program) |>                      # group by programme
  summarise(
    avg_score  = mean(score),               # calculate mean score
    n_students = n()                        # count students
  ) |>
  arrange(desc(avg_score))                  # sort by highest average

print(summary_stats)
```

Key dplyr verbs: `filter()` → `select()` → `mutate()` → `summarise()` → `arrange()`

> The pipe operator `|>` reads like a sentence: *"Take students, then filter, then group, then summarise."*

Exercise: Translate R Code into Python

Task: Translate the following R code into Python using `pandas`.

```r
# R Code — Translate this into Python
library(dplyr)

sales <- data.frame(
  product = c("A", "B", "C", "D"),
  revenue = c(15000, 8500, 22000, 11000),
  units   = c(120, 85, 200, 95)
)

result <- sales |>
  mutate(price_per_unit = revenue / units) |>
  filter(revenue > 10000) |>
  arrange(desc(revenue))

print(result)
```

Hints:

R (`dplyr`)	Python (`pandas`)
`mutate(col = expr)`	`.assign(col = expr)`
`filter(condition)`	`.query("condition")`
`arrange(desc(col))`	`.sort_values("col", ascending=False)`

Exercise: Model Solution

Python Solution:

```python
import pandas as pd

sales = pd.DataFrame({
    "product": ["A", "B", "C", "D"],
    "revenue": [15000, 8500, 22000, 11000],
    "units":   [120, 85, 200, 95]
})

result = (
    sales
    .assign(price_per_unit = sales["revenue"] / sales["units"])
    .query("revenue > 10000")
    .sort_values("revenue", ascending=False)
    .reset_index(drop=True)
)

print(result)
```

Key observations:

- `mutate()` → `.assign()` — adds a new computed column
- `filter()` → `.query()` — filters rows by condition
- `arrange(desc())` → `.sort_values(ascending=False)` — sorts descending
- Method chaining in pandas mirrors the pipe operator in R

Key Takeaways

Concept comparison at a glance:

Concept	R	Python
Assignment	`x <- 5`	`x = 5`
First element	`v[1]`	`v[0]`
Filter rows	`df[df$col > x, ]`	`df[df["col"] > x]`
Mean	`mean(v)`	`np.mean(v)`
Group & summarise	`group_by() |> summarise()`	`df.groupby().agg()`
Pipe / chain	`|>` or `%>%`	`.method()` chaining
Add column	`mutate()`	`.assign()`

Core takeaways:

- R and Python share the same **analytical logic** — only the syntax differs
- R's **1-based indexing** and `<-` assignment are the key adjustment points
- The `dplyr` pipeline in R mirrors **pandas method chaining** in Python
- For this course: **R is the primary language**; Python adds professional versatility
- Mastering one language makes learning the other significantly easier

Quiz Time:

Scan to Join the Game!

Quiz: Test Your Syntax Knowledge!

The Dataset:

ID	Sex	Age	Weight	Calories	Sport
1	F	21	48	1700	60
2	F	19	55	1800	120
3	F	23	50	2300	180
4	M	18	71	2000	60
5	M	20	77	2800	240
6	M	61	85	2500	30

```r
# R Code to create this dataset
df <- data.frame(
  ID = c(1, 2, 3, 4, 5, 6),
  Sex = c("f", "f", "f", "m", "m", "m"),
  Age = c(21, 19, 23, 18, 20, 61),
  Weight = c(48, 55, 50, 71, 77, 85),
  Calories = c(1700, 1800, 2300, 2000, 2800, 2500),
  Sport = c(60, 120, 180, 60, 240, 30)
)
```

Question 1: Filtering

R Code: `df %>% filter(sport > 100)`

What is the Python equivalent?

A. `df.filter(sport > 100)`

B. `df[df["sport"] > 100]`

C. `df.select("sport" > 100)`

D. `df["sport" > 100]`

Question 2: Adding Columns

R Code: `df %>% mutate(ratio = calories / weight)`

What is the Python equivalent?

A. `df["ratio"] = calories / weight`

B. `df["ratio"] = df["calories"] / df["weight"]`

C. `df.mutate(ratio = calories / weight)`

D. `df.add_column("ratio")`

Quiz Question 3: Grouping & Aggregating

R Code: `df %>% group_by(sex) %>% summarise(avg = mean(calories))`

What is the Python equivalent?

A. `df.groupby("sex").mean(calories)`

B. `df.groupby("sex")["calories"].mean()`

C. `df.mean(group="sex")`

D. `group(df, "sex")`

Quiz Question 4: Sorting

R Code: `df %>% arrange(desc(weight))`

What is the Python equivalent?

A. `df.sort(weight)`

B. `df.sort_values("weight", ascending=False)`

C. `df.order(weight)`

D. `df.sort(desc(weight))`

Quiz Question 5: Selecting Columns

R Code: `df %>% select(age, weight, calories)`

What is the Python equivalent?

A. `df[["age", "weight", "calories"]]`

B. `df.select(age, weight, calories)`

C. `df["age", "weight", "calories"]`

Conclusion and References

> *"Learn the concepts deeply in R — then map them to Python as needed."*

Conclusion:

- Both R and Python are powerful, **complementary** tools for data-driven decision-making
- Understanding syntax differences reduces friction when reading documentation or collaborating
- R remains the preferred language for **statistical rigour** and academic reporting in this course
- Python's versatility makes it a valuable addition to any data analyst's toolkit

References:

- Wickham, H. & Grolemund, G. (2017). *R for Data Science*. O'Reilly. r4ds.had.co.nz
- McKinney, W. (2022). *Python for Data Analysis* (3rd ed.). O'Reilly Media.
- R Core Team (2024). *R: A Language and Environment for Statistical Computing*. r-project.org
- Python Software Foundation (2024). *Python Language Reference*. docs.python.org
- Quarto (2024). *Revealjs Presentations*. quarto.org